

Lab 1: Reasoning in FOL

Lab exercises & submission

Francisco Lobo and Joseph Razavi

Academic session: 2024-25

The exercises of this assignment continue from the activities set for the lab sessions. If you haven't completed those, then go back and follow the *Lab 1 session activities* manual.

The activities manual describes the JSON specification format you must use to report parts of your solution (Vampire runs and proofs) to the summative exercises in this lab manual, and which is simply referred to as *proof spec* below. Your proof specs must give *all* the Vampire command-line options required to reproduce your results, including those necessary to make Vampire run in the conditions set by the exercise.

Remember that refutation proofs involve *only* the clauses which are part of the saturation process; not the formulas obtained during pre-processing and clausification. As you explore more complex proofs, you may find that the same *derived clause* is required in different parts of the proof tree. If this happens you do not need to repeat the sub-tree that derived this particular inference in various places; it is *sufficient* for the proof tree in your solution spec to only show once how each derived clause was obtained.

When specifying a resolution step in your proof specs you must report the full state of the saturation process when the inference is made, in addition to the premises, mgu, and conclusion of that particular application of the resolution rule. Note that the order in which you report your premises *must agree* with the order in which you specify the corresponding mgu substitutions.

Exercise 1

The idea of a *functional* binary relation is probably familiar to you; it asserts that objects are related (or mapped) to at most one thing (their image). In FOL **with equality**, we express that a binary relation R is functional as follows

$$\forall x, y, z. ((x R y \wedge x R z) \rightarrow y = z) \quad (1)$$

In the 1950s Jaques Riguet introduced the more general concept of *difunctional* relations. We can express that a binary relation R is difunctional as follows

$$\forall x, y. (x R y \leftrightarrow (\exists u, v. x R u \wedge v R u \wedge v R y)) \quad (2)$$

Mathematically, this simply asserts that R is the same as the triple composite $R ; R^* ; R$ where R^* denotes the converse of the relation R . In this exercise you'll use Vampire to show that Riguet's concept is indeed a generalisation of functional relations.

Develop a TPTP problem file using FOL with equality to prove that a binary relation that is functional (1) will also be difunctional (2). Your TPTP encoding must represent the binary relation R by a binary predicate `map`, which must be the **only symbol** of your signature. You need to save your code as `maps.p` and then use Vampire to obtain **3 different proofs**. For this exercise, you must obtain your proofs using the **plain given-clause algorithm**, so you need to enforce the corresponding Vampire options.

You must save your proofs as `maps-a.json`, `maps-b.json`, and `maps-c.json` in the proof spec format. Recall that two proofs are considered different *if and only if* each involves a resolution step which doesn't appear in the other. Thus each of your proof specs needs to include the details of a resolution step and corresponding state of the saturation algorithm, which shows that the proof is different from the other two. These must be supported by a corresponding map of formulas to indices, and in addition to the Vampire options used to generate the proof. For this exercise, your chosen resolution steps must have **non-trivial mgus**; this will mean that at least one of your premises cannot be a ground clause.

Mark scheme: There are 7 marks available for this exercise which are distributed as follows.

- 1 mark for your TPTP code, provided that your theory is equivalent to the reference solution
- 3 marks for correct tree specifications for your choice of proof options, provided these obtain 3 different proofs using the given-clause algorithm
- 3 marks for resolution steps unique to your choice of proof options, depending on how many of their details are correct

Exercise 2

The **TPTP Problem Library** is a comprehensive repository of axiom sets for various domains. One of these is called `SET001-0.ax` and included in your GitLab repo. It axiomatises membership and subset predicates for Set Theory and can be imported by your code using the TPTP directive

```
include('SET001-0.ax').
```

Once you add this line, you can translate mathematical notation to TPTP as follows.

- $a \in U$ meaning that a is an element of the set U can be written as `member(a,u)`
- $U \subset V$ meaning that U is a *strict* subset of V can be written as `subset(u,v)`
- $U = V$ meaning that the sets U and V are equal can be written as `equal_sets(u,v)`

In this exercise you will use the `SET001-0.ax` axiom set to produce a proof of a mathematical result which is relevant for the Description Logic part of the course.

Task A: Develop a first-order theory **without equality** that for a binary relation R encodes the following operators:

- The *direct image* of a set A across the relation R is defined by

$$\text{dirimg}(A) = \{y \mid \exists x. (x \in A \wedge x R y)\} \quad (3)$$

that is the set of all y 's related by R to some x in A

- The *value restriction* to a set B of the relation R is defined by

$$\text{valres}(B) = \{x \mid \forall y. (x R y \rightarrow y \in B)\} \quad (4)$$

that is the set of all x 's only related by R to y 's in B

Your TPTP encoding must represent the binary relation R by a binary predicate `role`, the direct image operator by a function symbol `dirimg`, and the value restriction operator by a function symbol `valres`.

Task B: Translate the following mathematical statement and add it as a conjecture to your code.

$$\forall A, B. (\text{dirimg}(A) \subseteq B \leftrightarrow A \subseteq \text{valres}(B)) \quad (5)$$

Note that the statement uses the (more common) reflexive subset relation $\subseteq$ and not the irreflexive subset relation $\subset$ which is defined in the `SET001-0.ax` axiom set.

The TPTP file that you develop in Tasks A & B needs to be saved as `sets.p` and you must also produce a file called `sets.cnf` with clausified version of the problem.

Your FOL theory will extend the signature of the `SET001-0.ax` axiom set with the encoding symbols specified above. In this exercise you are allowed to **extend the signature** further with additional **predicate symbols**. However, you can *only* do this using well-formed TPTP **definition** declarations; these will be processed separately from the rest of your theory.

Task C: Use Vampire to produce a proof of the above mathematical statement (5). Tune your Vampire options to make sure you obtain a refutation for both `sets.p` and `sets.cnf` in optimal time. Record your Vampire run as a `sets.json` proof spec. This must provide full details of a resolution step in the refutation you obtained for `sets.p` (including a map of the *required* formulas to indices). For this exercise, your proof spec **does not need** to include the details of the refutation proof tree that you produced. There is **no restriction** on which Vampire features you can use, but you must ensure Vampire clausifies your `sets.p` correctly with the run options that your proof spec sets.

Mark scheme: There are 6 marks available for this exercise which are distributed as follows.

- 2 marks for the first-order TPTP encoding of the operators in Task A, depending on the equivalence of your theory with the reference solution
- 2 marks for correct clausal form of the mathematical conjecture in Task B, depending on its equivalence with the reference solution
- 2 marks for a correct resolution step of the refutation you produce in Task C and a Vampire run that outperforms `./run_vampire --preset otter` on the reference solution

Exploring model building

You should complete the activity in this section before attempting the final modelling exercise. The goal is to explore and understand what happens when Vampire cannot construct a refutation. Consider the following modified version of the problem in the Lab 1 session activities manual.

$$\left\{ \begin{array}{l} \forall x. (\text{happy}(x) \leftrightarrow \exists y. (\text{loves}(x, y))) \\ \forall x. (\text{rich}(x) \rightarrow \text{loves}(x, \text{money})) \\ \text{happy}(\text{giles}) \end{array} \right\} \models \text{rich}(\text{giles})$$

- 🐼 Translate the problem to TPTP and saved it as `rich.p` (this will be a modified version of your `greed.p` file).

```
$ ./run_vampire rich.p --preset discount -s 1
% SZS status CounterSatisfiable for rich
% # SZS output start Saturation.
cnf(u14,axiom,
5   ~rich(X0) | loves(X0,money)).

cnf(u16,negated_conjecture,
   ~rich(giles)).

10 cnf(u17,axiom,
    loves(giles,sK0(giles))).

cnf(u13,axiom,
   ~loves(X0,X1) | happy(X0)).
15 cnf(u15,axiom,
    happy(giles)).

cnf(u12,axiom,
20   ~happy(X0) | loves(X0,sK0(X0))).

% # SZS output end Saturation.
% -----
```

Figure 1: Solving `rich.p` with `--preset discount -s 1`

If we try to use ordered resolution on this new problem, Vampire gives the output shown in Figure 1. It reports a `CounterSatisfiable` problem status because a refutation was not found; which means that the saturation algorithm ended without deriving the empty clause. When this happens Vampire outputs the saturated set of `cnf` clauses it obtained. As no contradiction was found, there are models that satisfy the axioms **and also** the negated conjecture. Vampire will highlight this reporting its termination reason was finding a saturated set of `Satisfiable` clauses.

Note: If this is confusing then you should revise the lecture materials to convince yourself of the following. The problem (axioms entailing the conjecture) will be *counter-satisfiable* precisely when the set of axioms and negated conjecture is *satisfiable*.

Before we accept Vampire's output status and conclude that the negated conjecture is consistent with the axioms of `rich.p`, it's reasonable to ask if changing, say, the clause and literal selection strategy that Vampire uses would find something more about this new problem...Possibly even reveal that there is a refutation after all !?

✚ You can check this using unordered resolution with the command

```
$ ./run_vampire rich.p --preset discount -s 0
```

Selecting of all literals in resolution ensures that Vampire computes all possible inferences for the problem. You should trace the saturation process using `--verbose gc` to confirm this, and think about why setting `--preset gc` is undesirable.

The result of unordered resolution is a larger saturated set, but you'll find that the status for the problem continues to be `CounterSatisfiable`. This example exhibits the fact that ordered resolution is *refutationally complete*, meaning that if refutation is possible then it will be found. This is an important property that we will discuss in the lectures.

We've convinced ourselves that `rich.p` is counter-satisfiable, but to prove this mathematically we still need to construct a model of the axioms that also satisfies the negated conjecture. Vampire can help us with model building if we disable the equality proxy and select `fmb` as the saturation algorithm. This *finite model building* process works by translating the problem of finding a model of a particular size into a SAT problem and iterating this process for larger model sizes.

✚ Try running the command

```
$ ./run_vampire rich.p -ep off -sa fmb
```

You should get the output shown in Figure 2 which gives a finite model making the axioms true but the conjecture false.

Vampire uses typed first-order formulas to give finite models, so it outputs TPTP code in `tff` form. This *typed logic* is outside the scope of this course unit, but you should still be able to read and understand all the `axiom` declarations, while mostly ignoring the `type` declarations.

Let's go through the output in Figure 2. You first read that Vampire tries and succeeds in finding a model of size 1, if that wasn't possible Vampire would try to make a model of size 2, and so on...Since a model exists, Vampire reports a `CounterSatisfiable` status in line 4, and then the `FiniteModel` is written from line 5 onwards.

The signature of our problem consists of

- 2 constant symbols `money` and `giles`
- 2 unary predicates `happy` and `rich`
- 1 binary predicate `loves`

for which we must give an interpretation. The *domain* of the model corresponds to the type `$i` and the booleans correspond to the type `$o`. The domain has a just 1 element and Vampire has identified it with the symbol `money`, the axiom declaration of line 8 asserts that everything in the domain is this element. The `giles` constant is also mapped to `money` by the axiom declaration of line 18.

```

$ ./run_vampire rich.p -ep off -sa fmb
TRYING [1]
Finite Model Found!
% SZS status CounterSatisfiable for rich
5 % SZS output start FiniteModel for rich
tff(declare_$i,type,$i:$tType).
tff(declare_$i1,type,money:$i).
tff(finite_domain,axiom,
10     ! [X:$i] : (
        X = money
    ) ).
tff(declare_bool,type,$o:$tType).
tff(declare_bool1,type,fmb_bool_1:$o).
tff(finite_domain,axiom,
15     ! [X:$o] : (
        X = fmb_bool_1
    ) ).
tff(declare_giles,type,giles:$i).
tff(giles_definition,axiom,giles = money).
20 tff(declare_happy,type,happy: $i > $o ).
tff(predicate_happy,axiom,
        happy(money)
    ).
tff(declare_loves,type,loves: $i * $i > $o ).
25 tff(predicate_loves,axiom,
        loves(money,money)
    ).
tff(declare_rich,type,rich: $i > $o ).
tff(predicate_rich,axiom,
30     ~rich(money)
    ).
% SZS output end FiniteModel for rich
% -----

```

Figure 2: Building a model for the `rich.p` problem

The interpretation of the predicate symbols is given as axioms declaring what is true in the model, in particular

- `happy(money)` at line 21
- `~rich(money)` at line 29
- `loves(money,money)` at line 25

In other words, `happy` is always true, `rich` is always false, and `loves` is always true.

- ✚ You should convince yourself that a structure with these characteristics does indeed satisfy the axioms but not the conjecture of the `rich.p` problem.

Of course, a model with a single element for this problem is not very interesting. Also, it doesn't distinguish between the constants `money` and `giles` which we instinctively want. But if we want Vampire to make models that distinguish between constants, we must add axioms which declare all constants to be pairwise different. This can be tedious so as a shortcut for it TPTP has the keyword `$distinct`.

- ✚ Try adding the axiom

```
fof(x, axiom, $distinct(money,giles,joe,francisco)).
```

to the problem and re-run the finite model building process, Vampire will construct a model with size 4 that proves `rich.p` is counter-satisfiable. You should make sure that you can reconstruct the model from the resulting `tff` output.

Exercise 3

The **TPTP Problem Library** contains a large number of ready-made problems in various domains. One of these is called `PUZ031+1.p` and included in your GitLab repo. It formalises a logical puzzle known as *Schubert's Steamroller* which in the 1980s was famously too difficult to solve, due to state space explosion during the saturation process. Indeed, if you try to solve the puzzle with unordered resolution

```
$ run_vampire PUZ031+1.p -s 0
```

you'll find that Vampire times out! You should compare this with the time taken to find a refutation using ordered resolution. Read the Schubert's Steamroller puzzle in `PUZ031+1.p` and study how the puzzle was modelled in first-order logic. In this exercise you'll do the same for a different puzzle, which you'll then be able to solve with the help of Vampire.

Task A: The following puzzle is taken from the book "**Arithmetical, Geometrical and Combinatorial Puzzles from Japan**" by Tadao Kitazawa (page 111).

In the Pets' Quality Resort, rooms #1 to #6 are in a row in that order on the ground floor. Rooms #7 to #12 are directly above rooms #1 to #6 respectively. Each room can accommodate one animal, and has its own light. At night, an animal who is not nervous turns off the light and sleeps well. An animal who is nervous leaves the light on and sleeps intermittently.

During the weekend, only the ground floor is open. Room #6 is occupied by the pet hamster of the part-time caretaker. The hamster is never nervous. Each of rooms #1 to #5 is occupied either by a dog or a cat. All five check out after a one-night stay.

Problem 10

On Sunday, a dog is nervous if and only if there are other dogs in both adjacent rooms. A cat is nervous if and only if there is another cat in at least one adjacent room. It is observed that only one room remains lit. How many cats are there in the P.Q.R. ?

Develop a first-order theory that formalises Kitazawa's Problem 10, as written above¹. Your signature must include the following symbols with the given semantics.

- 6 constant symbols `r1`, `r2`, `r3`, `r4`, `r5`, `r6`, which represent the ground floor rooms
- 1 binary predicate `next` representing the ordering of the rooms, such that `next(X,Y)` means that `Y` is the room after `X`
- 3 unary predicates `cat`, `dog`, `hamster` for the possible animals that can occupy a room, so that `cat(X)` means that room `X` is occupied by a cat, etc.
- 1 unary predicate `lit` that is only true for rooms with the light on

Do not represent the first-floor rooms of the P.Q.R. in your formalisation. Your TPTP code must be saved as `pets.p`. As in the previous exercise, you are allowed to **extend the signature** further with additional **predicate symbols**, provided these are introduced using well-formed TPTP definition declarations.

Task B: Using Vampire or otherwise give all the solutions of the puzzle from Task A, as finite models in the `tff` form introduced in the previous section. You must save these in as many letter indexed files `pets-a.tff`, `pets-b.tff`, ..., `pets-z.tff`, as you need. There are less than 26 solutions so you shouldn't have to create all of these files. Each of your files must give a unique solution. Only the `tff` axiom declarations in your finite models will be tested.

¹ In May 2022, a modified version of this puzzle appeared as part of **Alex Bellos's Monday puzzle** in The Guardian newspaper.

Mark scheme: There are 7 marks available for this exercise which are distributed as follows.

- 2 marks for the correct semantics of the TPTP signature, depending on your code being consistent with the semantics given above; for example, we may test that your predicate `next` isn't transitive
- 3 marks for the first-order TPTP theory for the puzzle, depending on the equivalence of your theory with the reference solution, and whether Vampire can find a model for it when run as `./run_vampire -ep off -sa fmb`
- 2 marks for all the `tff` finite models that solve the puzzle, which will be tested for consistency with the reference solution

Submission

Please follow the `README.md` instructions in your `COMP24412_2024` GitLab repo. Refresh the files of your `lab1` branch and develop your solution to the lab exercise by following the instructions of this lab manual. The solution to this lab consists of the following files.

Exercise 1: `maps.p`, `maps-a.json`, `maps-b.json`, and `maps-c.json`

Exercise 2: `sets.p`, `sets.cnf`, and `sets.json`

Exercise 3: `pets.p` and `pets-a.tff`, `pets-b.tff`, ...

These must be submitted to your GitLab repo and tagged as `lab1_sol`. You will find a skeleton proof spec file called `template.json` in your repo, which you can adapt for your solutions to Exercises 1 and 2. The `README.md` instructions that accompany the lab files include the `git` commands necessary to commit, tag, and then push **both** the commit and the tag to your `COMP24412_2024` GitLab repo. Links to further instructions on coursework submission and using `git` are also provided.

The deadline for submission is **18:00 on Tuesday 4th March**. The lab will be **auto-marked** offline. The auto-marker program will download your submission from GitLab and test it against reference solutions.

- The equivalence of your TPTP code with the reference solution will be tested by clausifying both and using Vampire to confirm that each clause is entailed by the other theory. Depending on how you encode the problems Vampire may **or may not** obtain these proofs within the default time limit. The consistency of your `tff` finite models will be tested in the same way.
- Your proof specs will be loaded as the `spec2latex.py` script does to test their correctness.

Your TPTP code will be tested by running the same Vampire version as the one installed on the Kilburn lab machines. Vampire will be run using the options you define in your proof specs, but the auto-marker will **always reset** the `--time_limit` option to the default of 1 minute. You need to make sure that your TPTP code *solves* the exercises within this time limit.

Important: Your TPTP `definition` declarations will be tested with the same logic as Vampire's `-updr` on setting to make sure they are well-formed. If these aren't well-formed, they will be considered regular axioms (which the reference solution is unlikely to validate).

Note: Extending the logical signature in Exercises 2 and 3 can be used to focus the proof search and ensure Vampire terminates within the time limit. But note that additional predicates may also cause the proof search to diverge, so you need to verify if your signature extensions have the desired effect.

Important: Every file in your submission should only contain printable ASCII characters. If you include other Unicode characters, for example by copying and then pasting code from the PDF of the lab manuals, then the auto-marker is likely to reject your files.